Benchmark Report of Urban Data Platform

Yixin Xia, Xinyue Feng, Hsi-Jui Chang

September 19, 2026

1 Benchmark Methodology

The platform was evaluated on the processed Delta tables and the `integrated_taxi_trips` table containing 9.4 million trips. The analytical notebook first recorded one end-to-end execution time for every query. These values identify expensive workload components, but they are not used to calculate optimization speedups because the first query also includes cold JVM, Delta metadata, and file-cache costs.

Each controlled optimization experiment used one discarded warm-up followed by five measured repetitions; the median is reported. Before and after variants were executed in the same environment and fully materialized with `collect()`. Result equality was checked using Spark `subtract()` in both directions. All experiments produced zero differing rows.

Four techniques were investigated:

- partition pruning on a January 2024 version of zone demand;
- broadcast versus shuffled hash join for the 265-row zone lookup;
- caching the shared borough-hour aggregate used by two air-quality queries;
- Adaptive Query Execution (AQE) for three analytical queries.

Physical changes were verified using `EXPLAIN FORMATTED`. AQE was also measured in reverse order because its initial timings showed small gains without any corresponding plan change.

A separate scale experiment attached city identifiers 1–10 to ten identical logical copies of the January 2024 trips. The one-city and ten-city zone-demand aggregations each used one discarded warm-up and three measured repetitions. This experiment measures logical volume scaling only: it does not represent ten physical input datasets, city-specific distributions, different weather or time zones, or concurrent workloads. Correctness was checked by comparing every synthetic city's result with the original one-city aggregate.

2 Execution Times Comparison

Original analytical workload

Query output	Time (s)	Result rows
Monthly demand by zone	101.717	793
Zone weather-demand variation	23.920	20
Distance by weather	22.686	5
Peak hours by weekday	15.166	7
Air-quality demand bands	11.322	4
Monthly demand trend	9.002	8
Air-quality demand correlation	8.578	4

Table 1: End-to-end observations from the original sequential notebook run.

The 101.717 s first query is useful as an end-to-end observation but is not directly comparable with the warmed controlled medians below.

Controlled optimization experiments

Technique	Before (s)	After (s)	Ratio	Valid gain?
Partition pruning	0.2260	0.1801	1.25×	Yes
Broadcast join	1.2314	0.3449	3.57×	Yes
Caching	0.6080	0.1037	5.86×	Yes
AQE: monthly zone demand	0.4115	0.3922	1.05×	No
AQE: distance by weather	0.4041	0.2949	1.37×	No
AQE: zone/weather variation	0.9158	0.8001	1.14×	No

Table 2: Median of five repetitions. AQE ratios are timing observations, not validated improvements.

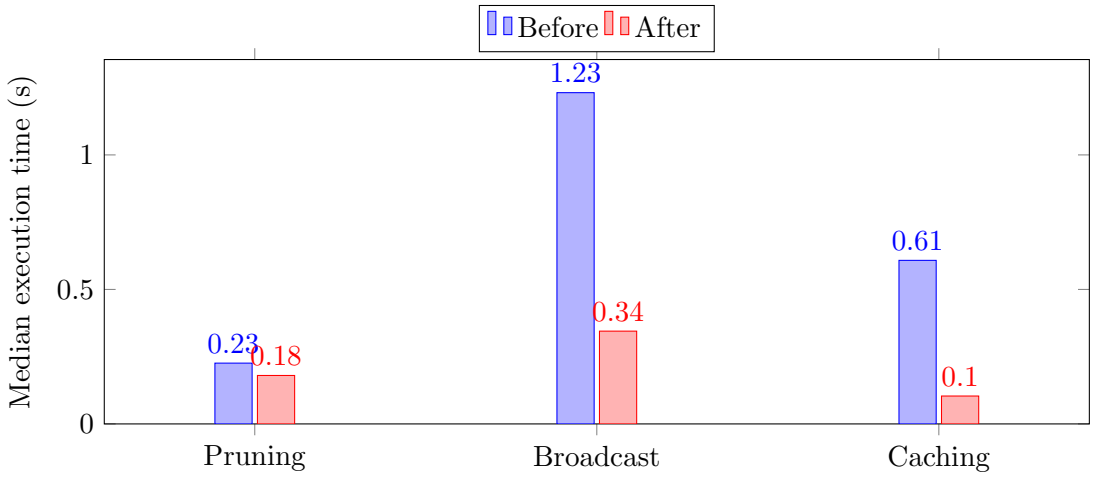


Figure 1: Validated before/after execution times.

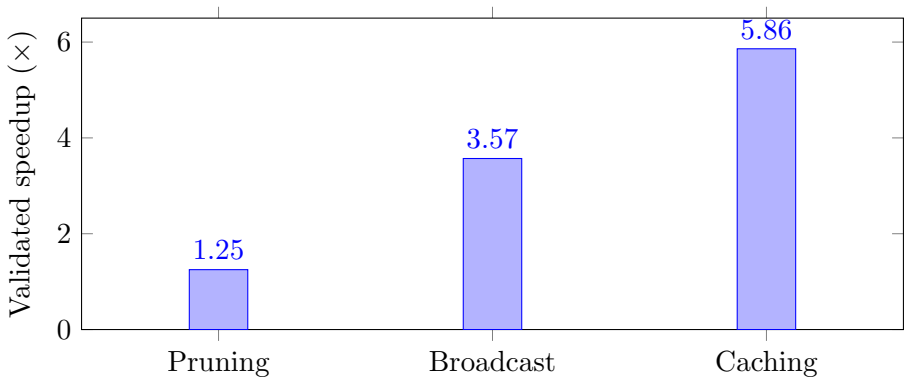


Figure 2: Only optimizations supported by both timing and plan evidence.

AQE order-effect check

Query	Off first	On second	On first	Off second
Distance by weather	0.4041	0.2949	0.4004	0.2756
Monthly zone demand	0.4115	0.3922	0.4687	0.3997

Table 3: Seconds. Reversing the order reversed the winner, showing that the second measurement benefited from warm state rather than AQE.

Synthetic ten-city scaling

Workload	Logical input rows	Median (s)	Throughput (rows/s)
One city, January 2024	2,926,993	0.9394	3,115,811
Ten city copies, January 2024	29,269,930	1.6773	17,450,623

Table 4: Synthetic scaling measurements. A 10× logical input increased latency by 1.79×.

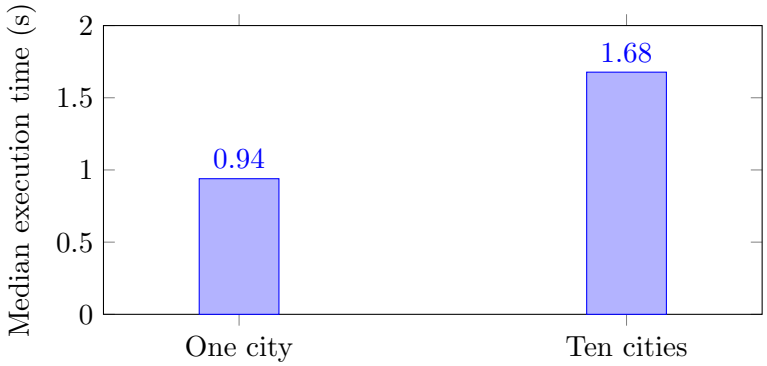


Figure 3: Measured latency for the one-month synthetic scaling workload.

Increasing the logical input from 2,926,993 to 29,269,930 rows increased the median execution time from 0.9394 s to 1.6773 s, a latency increase of only 1.79× for a 10× increase in logical rows. Measured throughput increased from 3,115,811 to 17,450,623 rows/s. The two directional result comparisons both returned zero differing rows.

This result should not be interpreted as true linear scaling across ten independent cities. The physical plan used a `BroadcastNestedLoopJoin` to attach ten city identifiers while reading the January source data once. Therefore, the experiment measures aggregation behaviour under logical row expansion, but does not reproduce ten physical datasets, additional input files, heterogeneous distributions, or concurrent city workloads.

Analytical-product storage

Delta product	Rows	Files	Size (MB)
Zone hourly demand profile	240,749	40	3.599
Origin–destination flow	82,847	40	2.053
Airport transfer summary	2,847	1	0.065
Weather-sensitive zone demand	3,930	1	0.065
Fare and tip patterns	131	1	0.010
Trip distance distribution	133	1	0.008
Total		84	**5.800**

Table 5: Active Delta data files. The products add 0.619% relative to the 936.188 MB integrated table.

All six products contained the required source, creation, refresh, and schema version metadata. The 5.800 MB overhead is small because each product stores aggregates rather than trip-level rows.

3 Analysis of Query Execution Plans

Technique	Before	After
Partition pruning	`PartitionFilters` absent; a row-level `Filter` follows the scan.	`PartitionFilters` contains year 2024 and month 1; the row-level filter disappears.
Broadcast join	`ShuffledHashJoin`; both inputs have hash-partitioning `Exchange` nodes.	`BroadcastHashJoin`; only the 265-row zone table has a `BroadcastExchange`.
Caching	Each query repeats the source scan, filter, and borough-hour `HashAggregate`.	Both downstream queries use `InMemoryTableScan`; the 6,435-row intermediate is computed once.
AQE	Static physical plan.	`AdaptiveSparkPlan` wrapper only; no `AQEShuffleRead`, and final plan equals initial plan.

Table 6: Relevant changes identified from `EXPLAIN FORMATTED`.

The first three techniques changed the physical work performed and also reduced median time. AQE changed only the wrapper around the plan. It did not coalesce shuffle partitions, handle a skewed join, or convert a join strategy. The reverse-order measurements therefore confirm that AQE produced no demonstrated performance improvement for these queries.

4 Performance Discussion

4.1 Largest improvement

Caching produced the largest validated gain, 5.86×, because the two air-quality analyses otherwise recomputed the same scan and borough-hour aggregation. Broadcast join was next at 3.57×: broadcasting 265 zone rows avoids redistributing millions of taxi rows. Partition pruning gave 1.25× because the month predicate reduced the read to one physical partition, although the local dataset is small enough for the absolute saving to remain modest.

4.2 Little or no effect

AQE had no measurable causal effect. The apparent winner followed measurement position when order was reversed, while the physical plan never adapted. Partition pruning also cannot help the full-history forms of the six analytical queries because those queries intentionally read every month.

4.3 Remaining expensive queries

Monthly zone demand remains expensive as a full-history scan, join, aggregate, and global sort. Zone weather-demand variation also remains computationally complex because it creates a zone-by-weather-hour grid, left joins hourly demand, performs several aggregations, and ranks zones. Peak-hour and monthly trend queries are single full-history aggregations. They have no reusable intermediate or small dimension join, so no artificial optimized variant was claimed.

4.4 Data characteristics

The results are explained by a large taxi fact table, a very small zone dimension, year/month physical partitioning, repeated borough-hour intermediates, and uneven demand across zones and weather categories. At this scale Spark's static choices were already adequate, leaving AQE nothing to change.

4.5 Trade-offs

Caching uses executor memory and must be invalidated after refreshes. Broadcasting replicates a table on every executor and is appropriate only while the dimension remains small. Partition pruning requires filters on the physical partition columns. The analytical products introduce 5.800 MB of storage, but this 0.619% overhead is small relative to the repeated aggregation work they avoid.

4.6 Expansion to ten cities

The synthetic experiment increased the monthly logical input from 2,926,993 to 29,269,930 rows, while median latency increased from 0.9394 s to 1.6773 s, or 1.79×. This demonstrates that the tested aggregation handled a 10× logical expansion efficiently on the current machine. However, because the source was scanned once and expanded through a broadcast cross join, this experiment alone does not establish that a distributed cluster is immediately required.

Before deployment, the experiment should be repeated with ten physically materialized city datasets, city-specific dimensions, concurrent queries, and representative data skew. Cluster expansion should then be based on measured latency, memory pressure, shuffle volume, and service-level requirements.

The controlled experiments support the following design recommendations. First, `city_id`, year, and month should be evaluated as partition or clustering keys for city-specific requests; monthly partition pruning already produced a measured 1.25× speedup. Second, small city and zone dimensions should remain eligible for broadcast joins, which reduced the tested join from 1.2314 s to 0.3449 s (3.57×). Third, shared aggregates should be cached when reused, since caching produced the largest validated improvement of 5.86×. Fourth, analytical products should be refreshed incrementally by city and month. Their current total storage overhead is only 5.800 MB, or 0.619% of the integrated table, indicating that materialized aggregates are inexpensive relative to repeated full-history computation.

File compaction and statistics collection should be retained as the number of city partitions grows. AQE should not be presented as an effective optimization until a future workload produces both a measurable improvement and an observable adaptive plan change.